ENTERPRISE TECHNICAL ARCHITECTURE & WORKING SPECIFICATION

Automated Voice AI Outbound Fee Collection System

Autonomous Outbound Dialer, Conversational Voice Bot & Disposition Reporting

Dataset Ingestion (Excel/PDF) -> Telephony (Twilio/Plivo/Exotel) -> LLM Reasoning -> Structured Output

Target Domain: Educational Institutes / Universities / EdTech
Status: Production-Grade System Architecture Blueprint

Architecture Version: 1.0 (Q4 2026)
Telephony: Twilio / Plivo / Exotel

1. Executive Summary & Problem Definition

Educational institutions, coaching academies, colleges, and schools face substantial operational friction in recovering overdue student tuition and term fees. Traditional collection relies on manual phone calls placed by front-office staff or accountants. This traditional approach suffers from high overhead, low dial volumes (an agent manages only 40-60 calls/day), lack of real-time structured data recording, and irregular follow-ups.

The proposed system solves this challenge end-to-end: it accepts tabular fee rosters from Excel spreadsheets or PDF ledger reports, parses parent and student records, and triggers automated, high-fidelity conversational voice calls. The voice agent states the pending balance and due date, requests a tentative payment date, or acknowledges prior payments. Responses are transcribed, parsed via LLM structured outputs into discrete schemas, and exported as updated spreadsheets and executive audit reports.

Core High-Level System Capabilities

- Multi-Format Data Ingestion: Automated extraction from Excel (.xlsx, .csv) and PDF fee dues tables.
- Programmatic Outbound Calling: High-throughput batch dialer supporting Twilio, Plivo, and Exotel.
- Human-Grade Spoken Conversation: Indian-accented English, Hindi, or bilingual natural language comprehension.
- Real-Time Speech Understanding: Fast STT + LLM reasoning for date entity extraction, disputes, and excuses.
- Automated Edge-Case Resolution: Handles answering machines, busy signals, wrong numbers, and installments.
- Closed-Loop Output Generation: Real-time database logging, updated Excel record generation, and summary PDF exports.

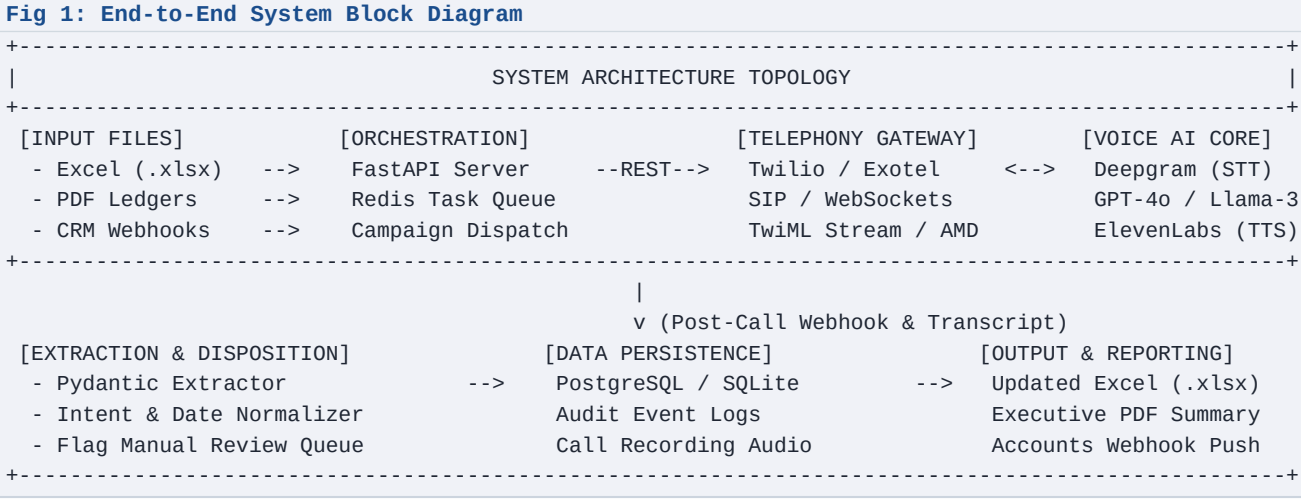
System High-Level Metrics & Targets

Metric / KPI	Industry Benchmark (Manual)	AI Voice System Target	Operational Impact
Dialing Speed	35 - 50 calls / staff / day	500 - 5,000+ calls / hour	100x throughput multiplier
Average Cost Per Call	Rs. 25.00 - Rs. 40.00 (Staff labor)	Rs. 1.20 - Rs. 2.50 (Telephony+AI)	90%+ cost reduction
Intent Extraction Accuracy	60-70% (Subjective notes)	94% - 98% (Structured LLM)	Uniform audit-ready records
Follow-Up Latency	2 - 5 days manual delay	Instant (Within 10 seconds)	Zero lag in accounts ledger

2. System Architecture & Component Decomposition

The system follows an event-driven, decoupled microservices architecture divided into five distinct operational tiers. This separation guarantees modularity, allowing independent scaling of telephony workers, LLM reasoning pipelines, and data export jobs without resource contention.

Fig 1: End-to-End System Block Diagram



Detailed Component Breakdown

- Tier 1 - Data Ingestion & Normalization: Accepts unstructured or tabular fee lists. Validates phone numbers to E.164 international format (+91XXXXXXXXXX), strips currency symbols, parses due dates into ISO-8601, and deduplicates records against active campaigns.
- Tier 2 - Campaign Dispatcher & Throttling Engine: Manages concurrent call limits to adhere to telecom carrier rate limits (e.g., 5-20 CPS - Calls Per Second). Implements regulatory time-window enforcement (e.g., calling strictly between 10:00 AM and 07:00 PM local time).
- Tier 3 - Telephony Gateway (SIP / WebSockets): Interfaces with Twilio, Plivo, or Exotel. Employs Answering Machine Detection (AMD) to identify human pickup vs. voicemail. When a human answers, it opens a bi-directional audio stream or TwiML Gather connection.
- Tier 4 - Real-Time Conversational AI Engine: Orchestrates Speech-to-Text (STT), conversational reasoning (LLM), and Text-to-Speech (TTS). Supports dynamic barge-in (caller interruption) so the parent can speak immediately without waiting for the bot to finish.
- Tier 5 - Extraction, Analytics & Reporting Engine: Parses call transcripts into structured JSON (payment status, tentative date, disputes). Updates the source Excel sheet with call outcomes and compiles a presentation-grade management PDF report.

3. Step-by-Step System Working & Operational Flow

The operational lifecycle of a fee collection campaign executes across seven synchronized phases. The diagram and detailed walkthrough below specify exact triggers, data transformations, and state transitions.

Fig 2: Campaign Lifecycle Sequence

```
[1. Input Upload]    --> Admin uploads 'fees_defaulters_october.xlsx' or 'dues_ledger.pdf'.
        |
[2. Validation]      --> Regex check phone numbers (+91), verify non-zero amounts, deduplicate.
        |
[3. Batch Dispatch]  --> Push contacts to Redis Queue. Throttled dispatcher triggers calls via Twilio/Exotel.
        |
[4. Pickup & AMD]    --> Answering Machine Detection: If voicemail -> drop reminder; If human -> connect AI.
        |
[5. Spoken Dialogue] --> Bot greets caller, presents pending fee & due date, asks for payment commitment.
        |
[6. AI Extraction]   --> LLM analyzes transcript: extracts outcome, tentative date, and dispute notes.
        |
[7. Output Delivery] --> System writes 'call_disposition_report.xlsx' and builds executive summary PDF.
```

Operational Stage Walkthrough

- Stage 1 - Ingestion & Schema Mapping: The admin uploads an Excel file with columns (Student_Name, Parent_Name, Mobile_Number, Amount_Due, Due_Date, Institute_Name). The parser validates each row. Missing phone numbers or invalid values are isolated in an error log.
- Stage 2 - Call Scheduling & Throttling: The dispatcher queries the queue and regulates dialing pace. If 1,000 parents are in the list, the system dials at a controlled concurrency (e.g., 10 parallel channels) to avoid telephony trunk saturation and ensure cloud server stability.
- Stage 3 - Call Pickup & Answering Machine Handling: When the call rings and connects, the telephony carrier's AMD algorithm analyzes the initial audio packet: if an automated voicemail or IVR is detected, the bot leaves a concise payment notice and terminates. If human voice is detected, the conversational session begins.
- Stage 4 - Dialogue Execution & Dynamic Prompting: The system dynamically injects row variables into the conversational prompt: 'Hello Mr. Sharma, this is an automated reminder from Apex Institute. We notice an overdue fee of Rs. 14,500 for Rahul Sharma, due on October 15th. Could you provide a tentative date when you will pay, or let us know if already paid?'
- Stage 5 - Response Capture & Classification: The parent's spoken response is streamed to the Speech-to-Text model. A structured LLM parser categorizes the response into one of five formal disposition codes (ALREADY_PAID, PROMISE_TO_PAY, DISPUTED, EXTENSION_REQUEST, or WRONG_NUMBER).
- Stage 6 - Database Update & Disposition Tagging: The system logs call metadata (Call SID, duration, timestamp, audio recording URL, full transcript, categorized outcome, and extracted tentative date) into a centralized audit ledger.

4. Spoken Dialogue Logic & State Machine

The conversational agent operates as a deterministic finite state machine (FSM) with dynamic LLM reasoning at each node. This ensures the bot never hallucinate non-existent fee figures while maintaining polite, natural dialogue.

Fig 3: Conversational Finite State Machine (FSM)



Dialogue Branching Scenarios & Handling

Branch 1: Parent Claims Already Paid

- Spoken Input: 'I already paid Rs. 14,500 yesterday evening through Google Pay UPI.'
- Bot Response: 'Thank you for updating us! We will verify this transaction with our accounts desk. Please retain your UPI receipt. Have a
- Extracted Output: { outcome: 'ALREADY_PAID', tentative_date: null, verification_required: true, payment_mode: 'UPI/GPay' }

Branch 2: Parent Promises Future Date (Promise to Pay)

- Spoken Input: 'My salary gets credited on the 7th of next month, so I will clear it by November 10th.'
- Bot Response: 'Understood. We have logged your tentative payment date as November 10th. An SMS confirmation has been sent. Than
- Extracted Output: { outcome: 'PROMISE_TO_PAY', tentative_date: '2026-11-10', notes: 'Salary credited on 7th' }

Branch 3: Fee Dispute / Financial Hardship / Installment Request

- Spoken Input: 'The amount is wrong, Rahul received a 30% merit scholarship which is not deducted.'
- Bot Response: 'I apologize for the discrepancy. I have flagged your record for immediate manual review by our finance counselor. They w
- Extracted Output: { outcome: 'DISPUTED', tentative_date: null, manual_follow_up: true, dispute_reason: 'Scholarship deduction pending

5. Data Schemas & Extraction Specifications

To guarantee high reliability and seamless integration with existing school management ERPs (e.g., Fedena, Salesforce Education Cloud, Tally, or custom portals), all inputs and outputs adhere to strict schemas.

1. Input Dataset Schema (Excel / CSV / PDF Table)

Field Name	Type	Required	Sample Value	Description
student_id	String	Yes	STU-2026-881	Unique student enrollment identifier
student_name	String	Yes	Rahul Sharma	Full name of the student
parent_name	String	Yes	Manoj Sharma	Name of primary guardian / parent
mobile_number	String	Yes	+919876543210	E.164 phone number to be called
pending_amount	Float	Yes	14500.00	Total unpaid overdue fee balance in INR
due_date	String	Yes	2026-10-15	Original fee deadline (ISO-8601 YYYY-MM-DD)
institute_name	String	Yes	Apex Academy	Name of school, college, or institute
course_class	String	Optional	Grade 11 - PCM	Class or academic batch

2. LLM Post-Call Structured Extraction Model (Pydantic)

Pydantic Schema for Guaranteed Extraction

```
from pydantic import BaseModel, Field
from typing import Optional, Literal

class CallDispositionExtraction(BaseModel):
    outcome: Literal[
        'ALREADY_PAID',       # Caller states fee is already paid
        'PROMISE_TO_PAY',     # Caller provided a future tentative payment date
        'EXTENSION_REQUEST',  # Caller asked for installment or financial extension
        'DISPUTED',           # Caller disputes amount or fees policy
        'WRONG_NUMBER',       # Recipient denies association with student
        'UNREACHABLE'         # Call failed, unanswered, or voicemail
    ] = Field(description='Primary call disposition outcome classification')
    tentative_pay_date: Optional[str] = Field(
        default=None, description='ISO date (YYYY-MM-DD) or explicit date mentioned by caller'
    )
    payment_reference_mode: Optional[str] = Field(
        default=None, description='Payment channel if already paid (e.g., UPI, NetBanking, Cash)'
    )
    summary_notes: str = Field(description='Concise 1-sentence synopsis of caller statements')
    manual_followup_flag: bool = Field(description='True if administrative intervention is required')
```

3. Output Report Schema (Exported to Excel & PDF)

Output Column	Source	Format	Analytical Utility
Call_Status	Telephony Gateway	COMPLETED / BUSY / NO_ANSWER	Telephony success analysis
Disposition_Code	LLM Extraction	PROMISE_TO_PAY / ALREADY_PAID	Financial pipeline forecasting
Tentative_Date	LLM Extraction	YYYY-MM-DD / N/A	Cash flow scheduling & follow-ups
Full_Transcript	STT Engine	Text String	Legal compliance & quality audit
Manual_Flag	Rule Engine	TRUE / FALSE	Filters prioritized queue for counselors

6. Open-Source GitHub References & Tech Stack Comparison

Rather than inventing voice streaming protocols from scratch, production systems leverage proven open-source voice orchestration engines. The table below analyzes the top GitHub repositories for this use case.

Framework	GitHub Repository	Architecture Focus	Recommended Application
Bolna	bolna-ai/bolna	Outbound Voice Campaigns	Top Pick: Dedicated outbound dialer with built-in intent e
Pipecat AI	pipecat-ai/pipecat	Real-Time Pipeline (WebSockets)	Best for ultra-low latency (<500ms) and dynamic caller ba
Twilio Agent	twilio/twilio-agent-connect-python	Twilio Native Connect	Official Twilio scaffolding for OpenAI Realtime & Convers
Deepgram Agent	deepgram-devs/deepgram-voice-agent	Telephony + AMD	Specialized in answering machine detection & voicemail
Vocode	vocodedev/vocode-python	Conversational State Agent	Python framework for phone calls, synthesis, and memor

Comparative Voice Stack Tradeoffs

Architecture Option A: Twilio <Gather speech> (Fastest to Deploy & Lowest Cost)

- Mechanism: Twilio handles STT natively via <Gather input='speech'> and posts transcript to a FastAPI webhook.
- Latency: 1.2s - 1.8s per turn (Turn-based half-duplex).
- Cost: ~$0.02 - $0.03 / min (Telephony) + $0.001 (LLM). Zero GPU infrastructure required.
- Verdict: Ideal for straightforward payment collection where the bot asks one question and records the answer.

Architecture Option B: Pipecat / Bolna WebSockets (Full Duplex with Barge-In)

- Mechanism: Twilio Media Streams send 8kHz mulaw audio over WebSockets to Deepgram Nova-2 -> LLM -> Cartesia TTS.
- Latency: 350ms - 600ms (Real-time natural conversation).
- Cost: ~$0.05 - $0.09 / min (STT + LLM + TTS + Telephony).
- Verdict: Recommended if callers frequently interrupt, ask complex fee questions, or require fluid human-like speech.

Architecture Option C: OpenAI Realtime / Gemini Live Telephony Bridge

- Mechanism: Direct speech-to-speech audio streaming through Twilio Media Streams.
- Latency: ~400ms end-to-end.
- Cost: ~$0.20 - $0.35 / min (Higher API pricing).
- Verdict: Excellent conversational nuance, but currently more expensive for high-volume automated debt campaigns.

7. Production Implementation Code Blueprint

Below is the complete, modular Python implementation for the FastAPI telephony controller, LLM extraction engine, and automated Excel/PDF output generator.

FastAPI Outbound Call Handler

```
# server.py - FastAPI Voice Telephony & Extraction Controller
import os
from fastapi import FastAPI, Form, Response
from twilio.twiml.voice_response import VoiceResponse, Gather
from openai import OpenAI
from pydantic import BaseModel

app = FastAPI(title='Fee Collection Voice Bot Server')
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

@app.post('/voice/start')
async def voice_start(student: str, amount: str, due: str, inst: str):
    vr = VoiceResponse()
    prompt = (
        f'Hello. This is an automated reminder from {inst}. '
        f'Pending dues of rupees {amount} were due on {due} for student {student}. '
        'Please specify when you will pay, or let us know if already paid.'
    )
    gather = Gather(
        input='speech', speech_timeout='auto', timeout=7, language='en-IN',
        action=f'/voice/process?student={student}&amount={amount}'
    )
    gather.say(prompt, voice='Polly.Aditi')  # Fluent Indian English
    vr.append(gather)
    vr.say('No response received. We will follow up later. Thank you.')
    vr.hangup()
    return Response(content=str(vr), media_type='application/xml')

@app.post('/voice/process')
async def voice_process(student: str, amount: str, SpeechResult: str = Form(default=''), CallSid: str = Form(...)
    vr = VoiceResponse()
    if not SpeechResult:
        vr.say('Thank you. Our team will contact you. Goodbye.')
        vr.hangup()
        return Response(content=str(vr), media_type='application/xml')

    # LLM Structured Extraction
    parsed = client.beta.chat.completions.parse(
        model='gpt-4o-mini',
        messages=[
            {'role': 'system', 'content': 'Extract fee payment commitment status and tentative date.'},
            {'role': 'user', 'content': f'Student: {student}, Due: {amount}. Caller said: {SpeechResult}'}
        ],
        response_format=CallDispositionExtraction
    ).choices[0].message.parsed

    # Dynamic Acknowledgment & Database Persistence
    record_call_outcome(CallSid, student, amount, SpeechResult, parsed)
    if parsed.outcome == 'ALREADY_PAID':
        msg = 'Thank you for informing us. We will verify and update our records. Goodbye!'
    elif parsed.outcome == 'PROMISE_TO_PAY':
        msg = f'Thank you. We have noted that you will pay on {parsed.tentative_pay_date}. Goodbye!'
    else:
        msg = 'Thank you. We have recorded your notes for our accounts counselor. Goodbye!'

    vr.say(msg, voice='Polly.Aditi')
    vr.hangup()
    return Response(content=str(vr), media_type='application/xml')
```

8. Legal Compliance, Telephony Regulations & Rollout Plan

Automated voice dialers are strictly regulated worldwide. Deploying this system requires strict adherence to telecommunications guidelines in India (TRAI / DLT) and global standards (US TCPA / FCC).

1. Indian Telephony Regulatory Framework (TRAI & DLT)

- DLT Registration (Distributed Ledger Technology): Under TRAI regulations, any commercial or institutional entity making automated calls or SMS in India must register their Principal Entity (PE) ID and Telemarketer (TM) ID on telecom DLT portals (e.g., Jio, Airtel, Vodafone-Idea DLT).
- Transactional Voice Category: Fee reminder calls are classified as Service Implicit / Transactional. This allows dialing registered students even if their numbers are listed on the National Do Not Disturb (NDND / NDNC) registry, provided a pre-existing student relationship exists.
- Permitted Calling Hours: Automated dialers must only operate between 09:00 AM and 08:00 PM local time. The campaign dispatcher must enforce automatic queue suspension outside these hours.
- Consent & Audio Recording Disclosure: The call opening should incorporate a short disclosure: 'This call is an automated notice from [Institute] and is recorded for quality and verification purposes.'

2. Multi-Phase Production Rollout Plan

Phase	Timeline	Milestones	Target Output
Phase 1: Ingestion & Parser	Week 1	Build Excel & PDF parsers, validate E.164 phone data	Clean tested JSON dataset
Phase 2: Telephony Prototype	Week 2	Set up Twilio/Exotel account, configure FastAPI webhooks	Test call to internal numbers
Phase 3: Real-Time AI Agent	Week 3	Integrate Deepgram/Polly + GPT-4o-mini structured parser	Sub-800ms intent recognition
Phase 4: Output Engine	Week 4	Build Excel updater & executive PDF generator	Closed-loop automated reports
Phase 5: Pilot & Scale	Week 5-6	DLT approval, pilot with 200 real dues, full batch rollout	Production scale (5k calls/day)

Summary Recommendation for Implementation

- Step 1: Use Exotel or Plivo if deploying primarily in India for native TRAI/DLT caller-ID compliance.
- Step 2: Start with the Twilio <Gather> / Bolna architecture for simplicity and rapid 2-week deployment.
- Step 3: Connect PostgreSQL to archive all call audio recordings, raw STT logs, and audit timestamps.
- Step 4: Schedule weekly automated PDF summary dispatches to the Chief Financial Officer (CFO) and Dean.